

# Neural Network and Deep Learning

Big Data Analytics

# Readings

- Neural Network and Deep Learning
  - Chap. 1

# Recap

Big Data Analytics

# Using Neural Nets to Recognize Handwritten Digits

504192

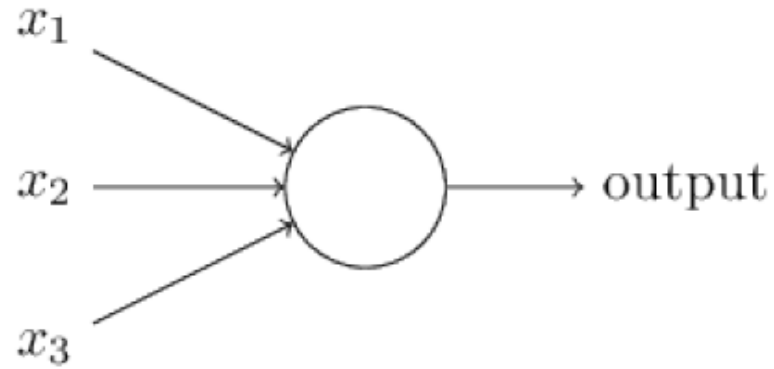
- How we recognize number 9
  - A loop at the top
  - A vertical stroke in the bottom right
- Not so simple to express this algorithmically

# Paradigm Shift of Computation

- Conventional Approach
  - Tell the computer what to do
  - Breaking big problems into many small and well defined tasks
- New Paradigm
  - Computer learns from observational data
  - Computer figures out its own solution to the problem

# Sigmoid Neurons

- Sigmoid neurons look like Perceptrons



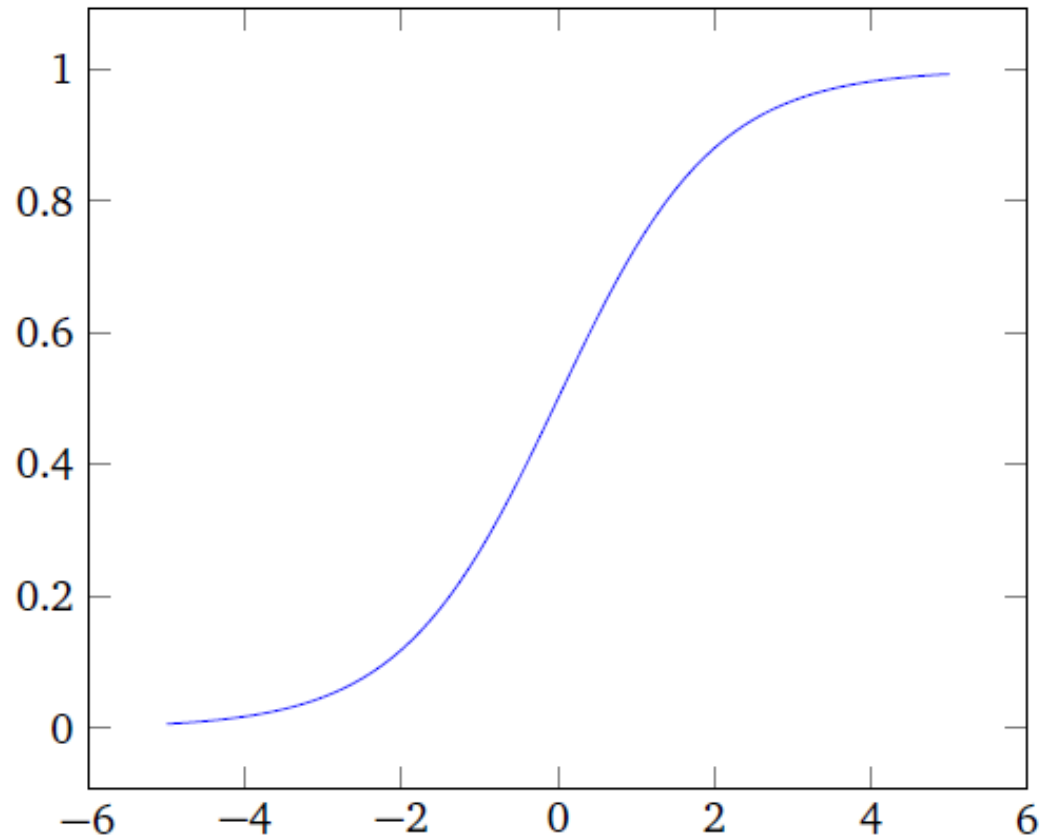
- Inputs can take on any values between 0 and 1
- Each input has weight

- Output is  $\sigma(wx + b)$  where  $\sigma(z) \equiv \frac{1}{1 + e^{-z}}$   $z = w \cdot x + b$

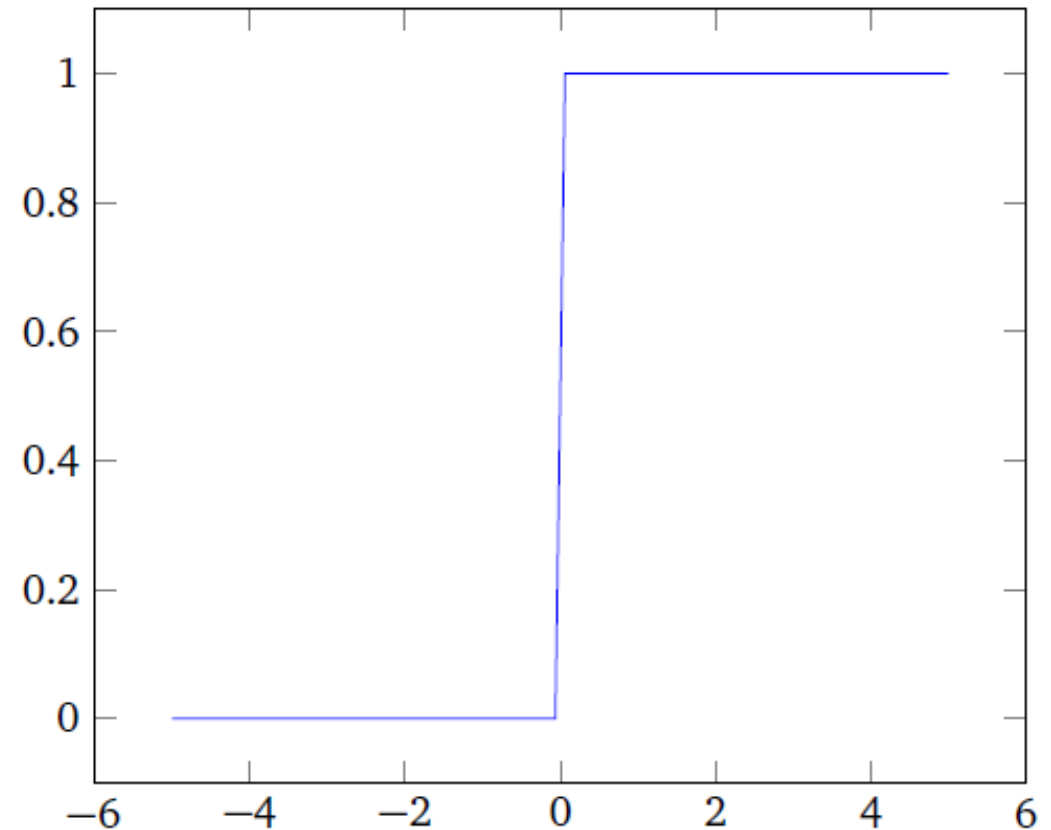
# Sigmoid Neurons: Smoothed version of step function

$$\sigma(z) \equiv \frac{1}{1 + e^{-z}} \quad z = w \cdot x + b$$

Sigmoid function



Step function



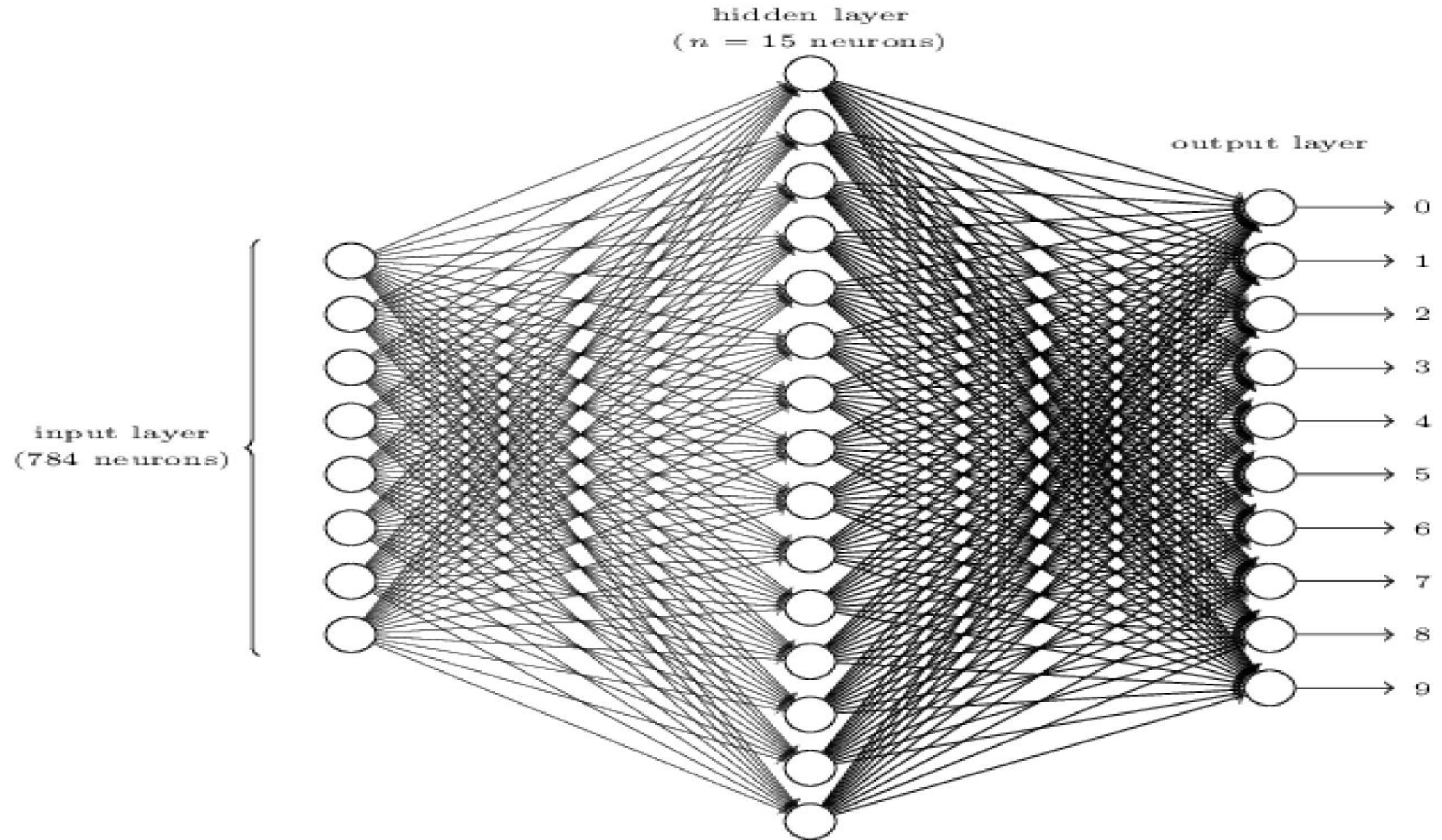
# Using Neural Nets to Recognize Handwritten Digits



- MNIST dataset
  - 28 X 28 grey scale
  - 60,000 training set and 10,000 test set
  - 250 people wrote training set and another 250 people wrote test set



# A simple network to classify handwritten digits

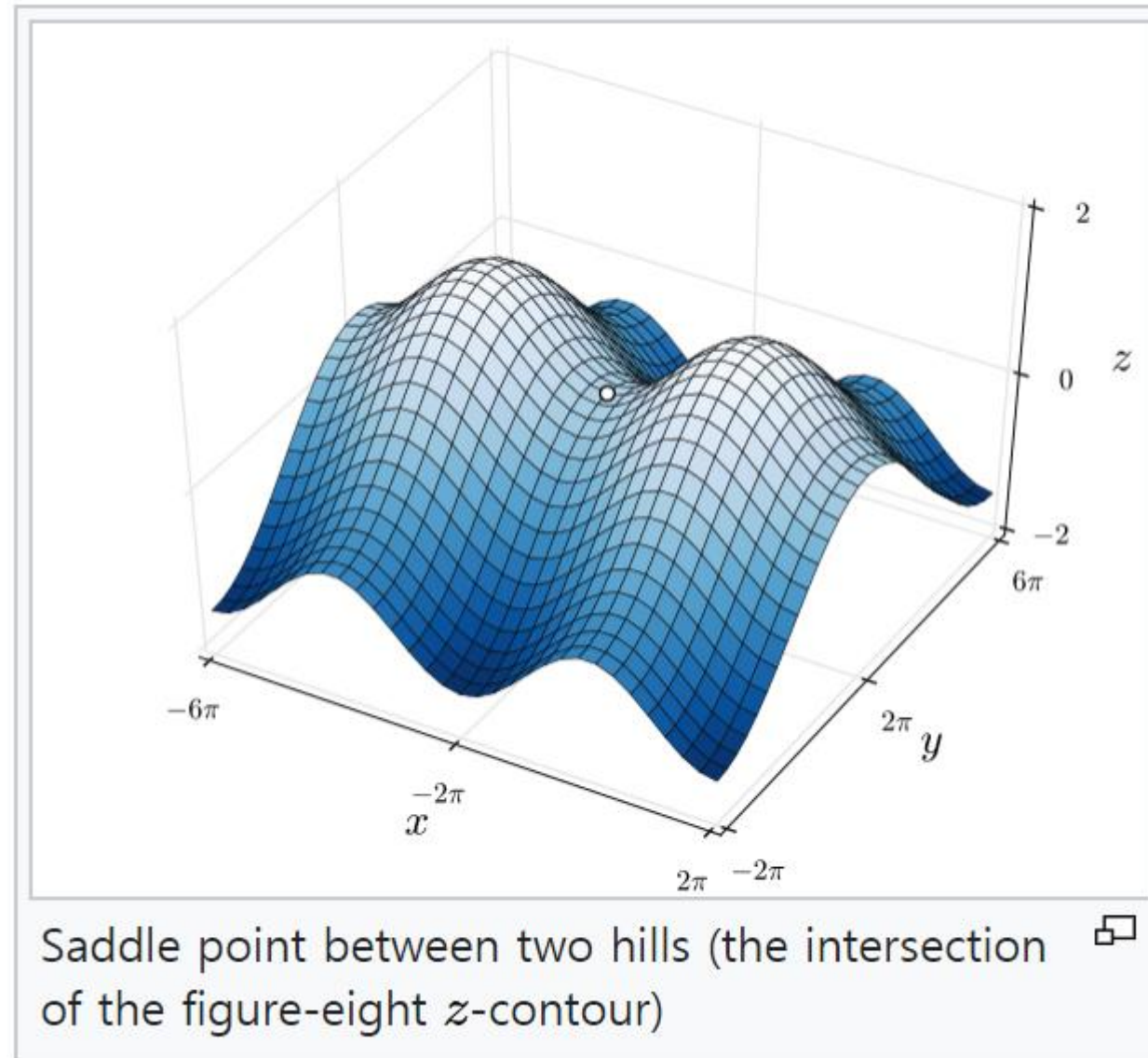


- Approximation of one variable function

- Approximation of two variable function

- Min/Max problems

- Min/Max problems



- Least Squares

2.[14 points] Least Squares (Linear Regression) Here is a dataset given as (x, y) and the cost function C:

$$\{(1, 2), (2, 4), (3, 5), (4, 6)\} \quad C = \sum_{i=1}^n (y_i - (a x_i + b))^2$$

(a) [8 Points] Write 2 linear equations in terms of 2 variables a and b to minimize the cost. Show all your work

**(b) [4 Points]** What are the values of the variables? Show all your work

**(c) [2 Points]** What is the predicted value of  $y$  when  $x = 10$ ?



- Gradient vector

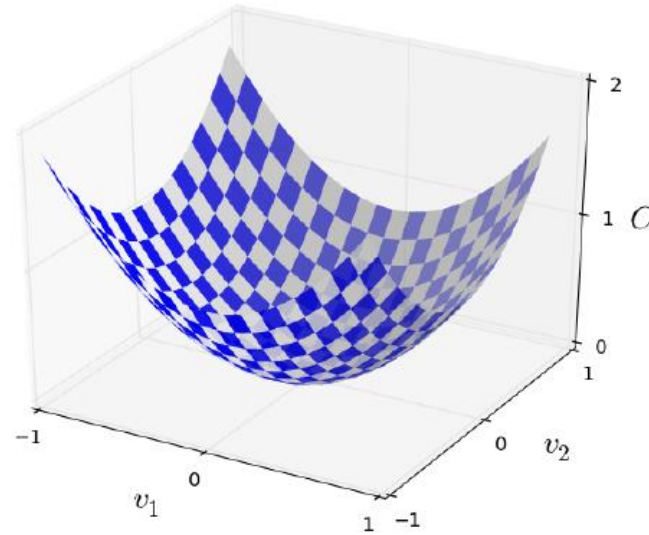
# Gradient descent

- Gradient descent is an iterative algorithm for finding the local minimum of a differentiable function
- Get a gradient vector of a cost function  $C$ ,  $\nabla C \equiv \left( \frac{\partial C}{\partial v_1}, \frac{\partial C}{\partial v_2} \right)^T$
- Decrease the input vector  $\Delta v = -\eta \nabla C$ , where  $\eta$  is a small positive parameter (learning rate)



# Learning with gradient descent

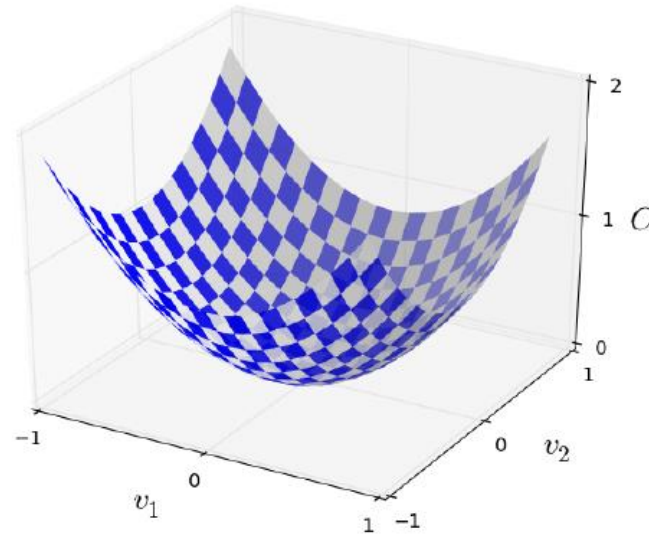
- Let's suppose we're trying to minimize some function  $C(v)$  where  $v$  is a vector  $v_1, v_2, v_3, \dots$
- And imagine  $C$  as a function of just 2 variables,  $v_1$  and  $v_2$



- Then  $\Delta C \approx \frac{\partial C}{\partial v_1} \Delta v_1 + \frac{\partial C}{\partial v_2} \Delta v_2$
- Therefore  $\Delta C \approx \nabla C \cdot \Delta v$  where  $\nabla C \equiv \left( \frac{\partial C}{\partial v_1}, \frac{\partial C}{\partial v_2} \right)^T$  and decrease  $\Delta v = -\eta \nabla C$

# Learning with gradient descent

- Let's suppose we're trying to minimize some function  $C(v)$  where  $v$  is a vector  $v_1, v_2, v_3, \dots, v_m$



- Then  $\Delta C \approx \nabla C \cdot \Delta v$  where  $\nabla C \equiv \left( \frac{\partial C}{\partial v_1}, \dots, \frac{\partial C}{\partial v_m} \right)^T$  and decrease  $\Delta v = -\eta \nabla C$   
$$v \rightarrow v' = v - \eta \nabla C$$

# Learning with gradient descent

- Input  $x$  is a  $28 \times 28 = 784$ -dimensional vector
- Desired output  $y(x) = (0, 0, 0, 0, 0, 0, 1, 0, 0, 0)^T$  if a particular training image  $x$ , depicts a 6
- The cost function  $C(w, b) \equiv \frac{1}{2n} \sum_x \|y(x) - a\|^2$  where
  - $w$  denotes the collection of all weights in the network
  - $b$  denotes all the biases
  - $n$  is the total number of training inputs
  - $a$  is the vector of outputs from the network when  $x$  is the input

# Learning with gradient descent

- $\Delta C \approx \nabla C \cdot \Delta v$  where  $\nabla C \equiv \left( \frac{\partial C}{\partial v_1}, \dots, \frac{\partial C}{\partial v_m} \right)^T$  and decrease  $\Delta v = -\eta \nabla C$
- Therefore  $v \rightarrow v' = v - \eta \nabla C$

- Now for weight  $w_k$  and bias  $b_l$

$$w_k \rightarrow w'_k = w_k - \eta \frac{\partial C}{\partial w_k}$$

$$b_l \rightarrow b'_l = b_l - \eta \frac{\partial C}{\partial b_l}.$$

# Learning with gradient descent

- $\Delta C \approx \nabla C \cdot \Delta v$  where  $\nabla C \equiv \left( \frac{\partial C}{\partial v_1}, \dots, \frac{\partial C}{\partial v_m} \right)^T$  and decrease  $\Delta v = -\eta \nabla C$
- Therefore  $v \rightarrow v' = v - \eta \nabla C$

- Now for weight  $w_k$  and bias  $b_l$

$$w_k \rightarrow w'_k = w_k - \eta \frac{\partial C}{\partial w_k}$$

$$b_l \rightarrow b'_l = b_l - \eta \frac{\partial C}{\partial b_l}.$$

- Where  $C(w, b) \equiv \frac{1}{2n} \sum_x \|y(x) - a\|^2$



# Learning with gradient descent

- $\Delta C \approx \nabla C \cdot \Delta v$  where  $\nabla C \equiv \left( \frac{\partial C}{\partial v_1}, \dots, \frac{\partial C}{\partial v_m} \right)^T$  and decrease  $\Delta v = -\eta \nabla C$
- Therefore  $v \rightarrow v' = v - \eta \nabla C$

- Now for weight  $w_k$  and bias  $b_l$

$$w_k \rightarrow w'_k = w_k - \eta \frac{\partial C}{\partial w_k}$$

$$b_l \rightarrow b'_l = b_l - \eta \frac{\partial C}{\partial b_l}.$$

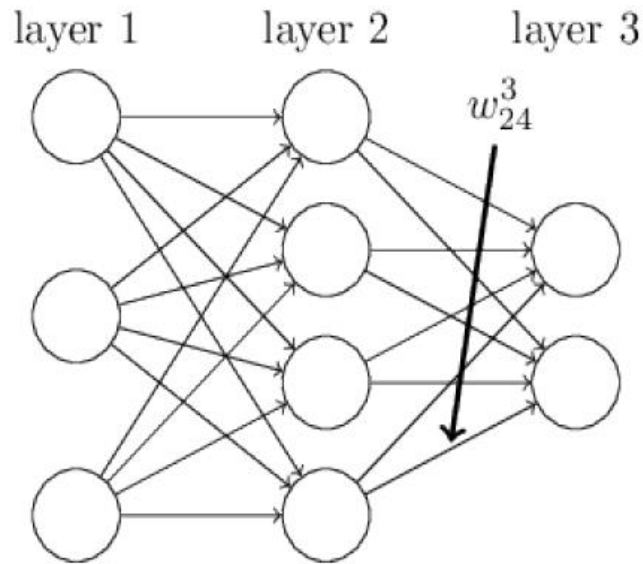
- Where  $C(w, b) \equiv \frac{1}{2n} \sum_x \|y(x) - a\|^2$

$$C_x \equiv \frac{\|y(x) - a\|^2}{2} \text{ and } C = \frac{1}{n} \sum_x C_x$$

# Matrix-based approach to compute the output

- The Notation

$$w_{jk}^l$$

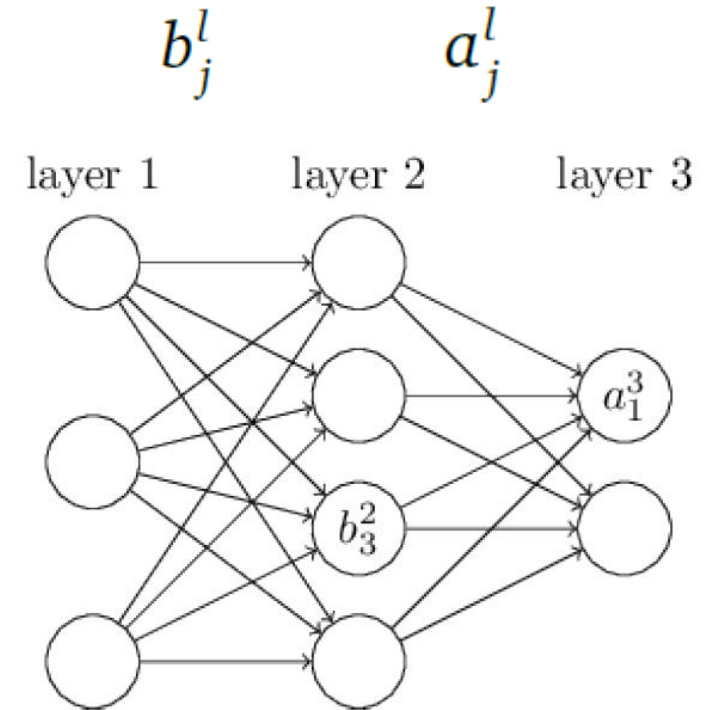


$w_{jk}^l$  is the weight from the  $k^{\text{th}}$  neuron in the  $(l-1)^{\text{th}}$  layer to the  $j^{\text{th}}$  neuron in the  $l^{\text{th}}$  layer

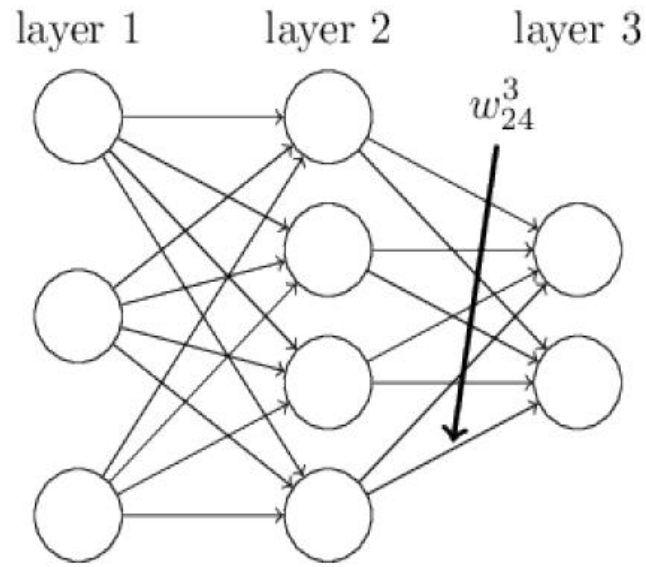
$$a_j^l = \sigma \left( \sum_k w_{jk}^l a_k^{l-1} + b_j^l \right)$$

$$a^l = \sigma(w^l a^{l-1} + b^l)$$

$$a^l = \sigma(z^l)$$



$$w_{jk}^l$$



$w_{jk}^l$  is the weight from the  $k^{\text{th}}$  neuron in the  $(l-1)^{\text{th}}$  layer to the  $j^{\text{th}}$  neuron in the  $l^{\text{th}}$  layer

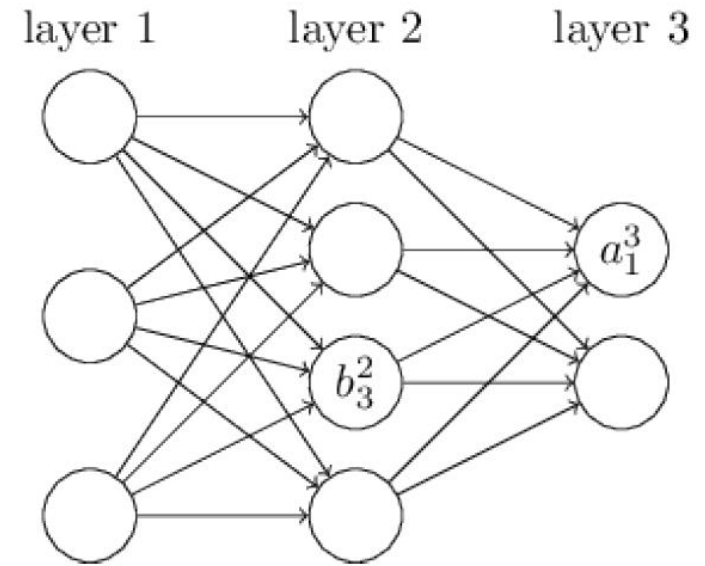
$$a_j^l = \sigma \left( \sum_k w_{jk}^l a_k^{l-1} + b_j^l \right)$$

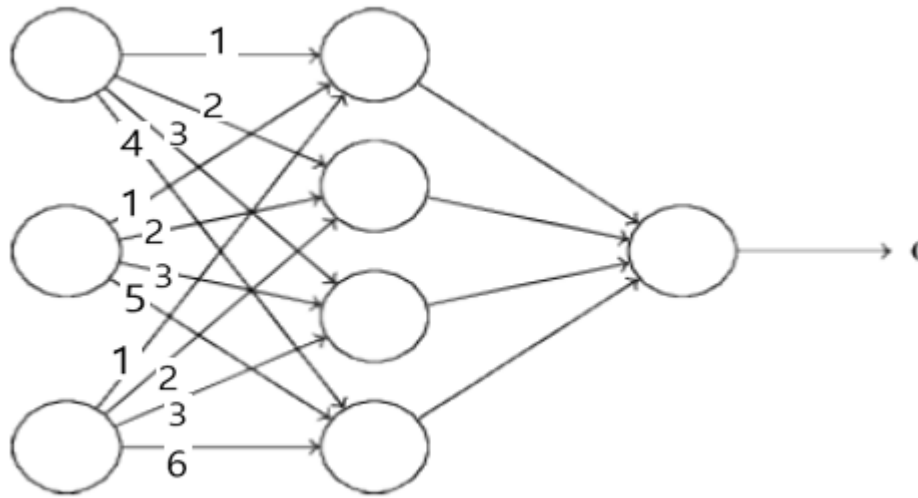
$$a^l = \sigma(w^l a^{l-1} + b^l)$$

$$a^l = \sigma(z^l)$$

$$b_j^l$$

$$a_j^l$$

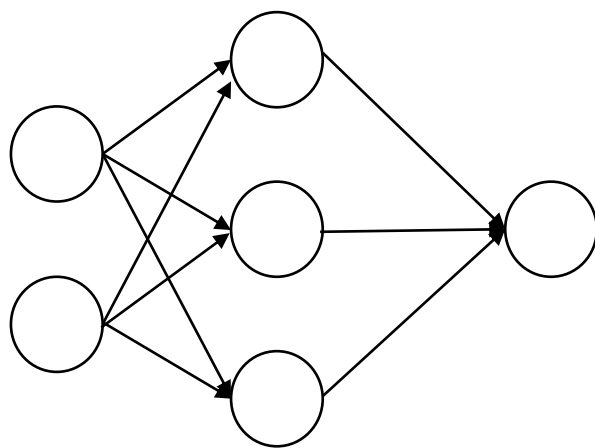




$w^2$  ?

$z^2$  ?

$a^1$  vector is  $(0.1, 0.1, 0.7)^T$  **and**  $b^2$  vector is  $(3, 4, 5, 6)^T$



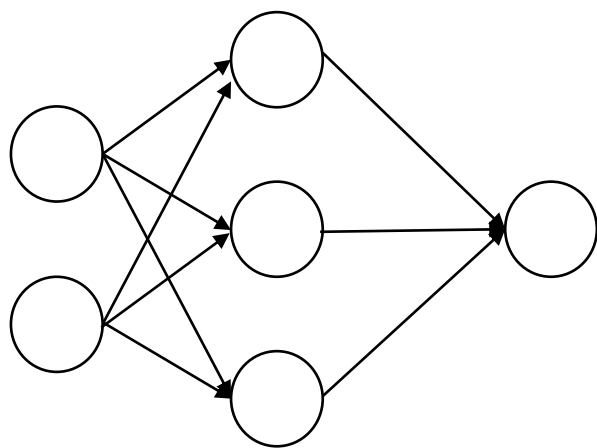
$$a^1 = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$$

$$W^2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ 1 & 1 \end{bmatrix}$$

$$b^2 = \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

$$W^3 = [1 \quad -1 \quad 1]$$

$$b^3 = [1]$$



$$a^1 = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$$

$$W^2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ 1 & 1 \end{bmatrix}$$

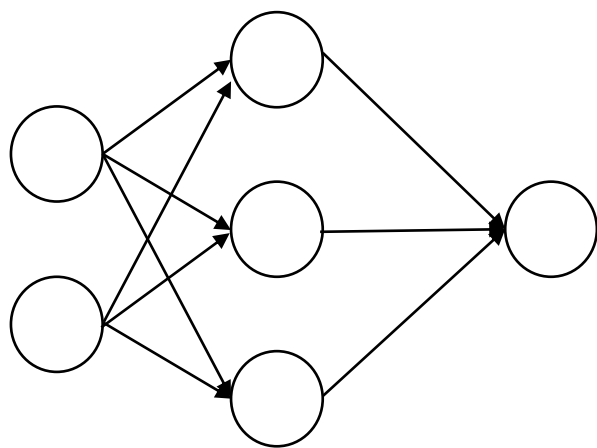
$$b^2 = \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

$$W^3 = [1 \quad -1 \quad 1]$$

$$b^3 = [1]$$

$$Z^2 = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ -2 \\ 2 \end{bmatrix}$$



$$a^1 = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$$

$$W^2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ 1 & 1 \end{bmatrix}$$

$$b^2 = \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

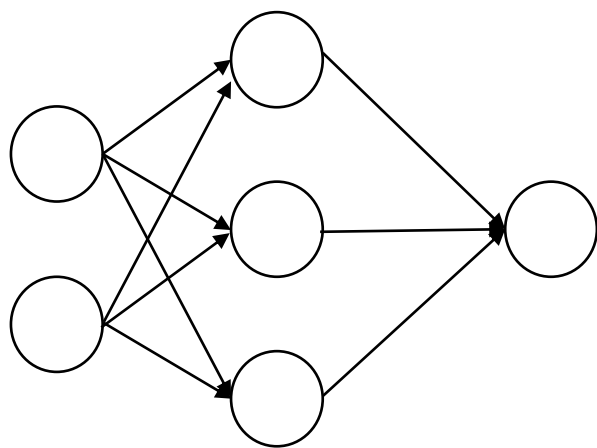
$$W^3 = [1 \quad -1 \quad 1]$$

$$b^3 = [1]$$

$$Z^2 = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ -2 \\ 2 \end{bmatrix}$$

$$a^2 = \begin{bmatrix} 0.5 \\ 0.119 \\ 0.881 \end{bmatrix}$$



$$a^1 = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$$

$$W^2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \\ 1 & 1 \end{bmatrix}$$

$$b^2 = \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

$$W^3 = [1 \ -1 \ 1]$$

$$b^3 = [1]$$

$$Z^2 = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ -2 \\ 2 \end{bmatrix}$$

$$a^2 = \begin{bmatrix} 0.5 \\ 0.119 \\ 0.881 \end{bmatrix}$$

$$Z^3 = [1.262 + 1] = [2.262]$$

$$a^3 = [0.906]$$



# Back Propagation